Possession without the ball — DSBA

Team DSBA — Feras Alsadadi, Talal Kamal, Ali Mohamed, Mohammed Hejairi

More than a Hack 2027, Phase 1: frame-level possession (`H` / `A` / `D` / `X`) for BAR-ATH.

In one paragraph. The ball is missing on 41% of graded frames, mostly at the moments that decide possession. We read possession from the team's shape instead. A team with the ball spreads wide and long; the team without it compacts. We train a ball-free model of that on **pseudo-labels** taken from the data itself. We combine it with ball proximity and a dead-ball detector in a **3-state HMM (H/A/D)** that pays a high price for switching teams. Every setting was chosen with a **ball-masking validation** that needs no ground truth: we hide the ball where we know the answer and check that the pipeline still finds it.

Reproduce

```
py -m venv .venv                                    # Python 3.12
.venv/Scripts/python -m pip install -r requirements-solution.txt
# unpack the data bundle into ./data  (data/tactical/BAR-ATH_tactical_atd.txt, ...)
.venv/Scripts/python run_pipeline.py                # -> evaluation/results/DSBA.csv, then validates it
```

- One command, about 65 s. The first run also parses and caches the tracking file in `data/tactical/.cache/`, which is gitignored.
- **Deterministic.** Two runs, and a fresh venv built from the pins, give the identical file (MD5 `97f61dfa580bb6f80156284c834b4873`).
- `python run_validation.py` re-runs the settings study below.
- `python synthetic/make_synthetic.py` builds a synthetic match in the same file formats. It is for development only (see the end of this document).

Assets used

asset	how
tactical ATD	everything: ball, proximity, team shape, speeds
Smart Tagging	set pieces → restart anchors; phases → weak team prior (direction learned); kickoff side
homography, videos, broadcast ATD	**not used**: the ATD is already in pitch coordinates, and the broadcast has no time alignment
public 5-min ground truth	**only** to check that frames line up and the file is well formed. No setting was chosen with it

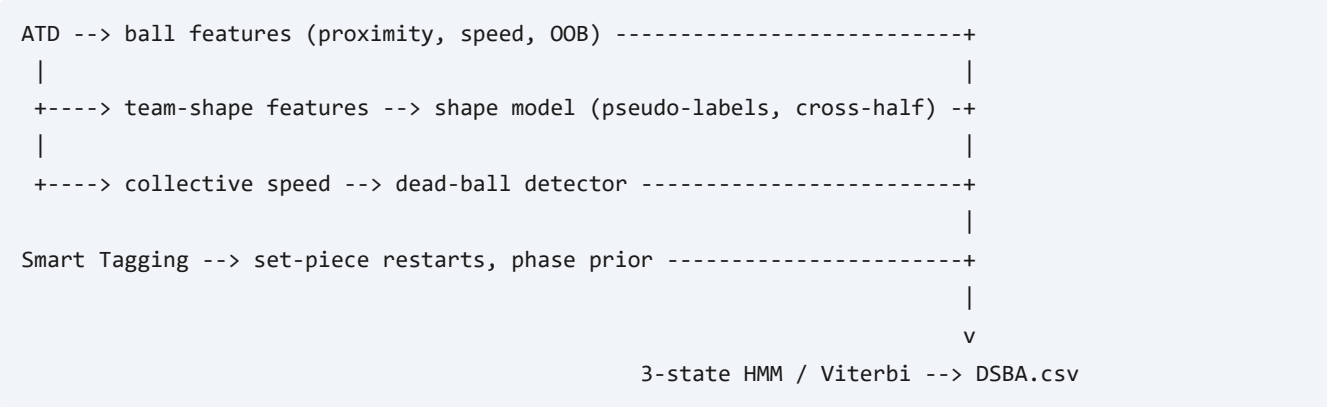
No hand labels, no human-produced data about this game, no external data.

Training data

dataset	source	labelled by	size
pseudo-labels (section 2)	the tactical ATD and Smart Tagging of this game	a fixed rule in `possession/pseudo.py`, no person	15,283 team frames, 16,759 dead frames

No other labelled data was used: no hand labels on any game, no ground truth (public or private) and no pre-trained models. The unsupervised parts (crowd-speed mixture, phase-tag direction) are fitted on this game's tracking without labels.

Pipeline



1. Ball features (`possession/features.py`)

- **Ball cleaning:** drop implausible and teleporting detections (faster than 40 m/s). Bridge occlusions of ≤ 0.4 s.
- **Proximity log-odds:** `clip(d_away − d_home, ±3)`, using each team's nearest player. It is scaled down when nobody is within ~2.5 m and when the ball is in flight (> 7 m/s). A loose ball therefore carries no evidence, and possession stays with the last team, as CHALLENGE.md asks.
- **Out of bounds:** the ball is seen more than 0.75 m outside the lines.
- **Collective speed:** median player speed, smoothed over 2 s.

2. Pseudo-labels (`possession/pseudo.py`)

Frames whose answer is obvious, chosen by a fixed rule, never by a person:

- **team:** the ball is observed and slow (< 5 m/s), a player is within 1 m of it, and nobody from the other team is within 3 m. That gives 15,283 frames (9,490 home, 5,793 away).
- **dead:** the ball is seen out of bounds, a tagged set piece before its estimated restart, or the graded frames before the published 2nd-half kickoff. That gives 16,759 frames.

3. Team-shape model (`possession/shape.py`, `possession/shape_model.py`)

- **Features.** There are 55 ball-free features per frame. Per team: centroid, width, length, spread, the height of the deepest outfield player, the most advanced player, players in the opponent half, speed, "push" (centroid velocity towards goal), and free space (mean distance to the nearest opponent). For each we also use home minus away, its 1 s average and its 2 s trend.
- **Orientation.** Everything is measured towards the team's own attacking goal. The attacking direction is found per half from the data (the deepest player sits at the goal the team defends), not hard-coded.
- **Model.** Gradient-boosted trees (`HistGradientBoostingClassifier`), trained **cross-half**: the model that predicts half 1 never saw half 1. Every prediction the pipeline uses is therefore out-of-sample.

held-out half	accuracy vs pseudo-labels	AUC	logistic-regression AUC
1	0.864	0.934	0.906
2	0.876	0.940	0.913

What it learned, by permutation importance (drop in AUC):

1. home-minus-away lateral spread (`diff_sy`)

2. home-minus-away distance from the centre line (`diff_cy_abs`)
3. home-minus-away length, 1 s average
4. home speed
5. away lateral spread
6. home-minus-away depth spread
7. the 2 s trend of the push towards goal

In plain words: **the team with the ball is the wider one**, which is the classic coaching idea of "make the pitch big in possession". The trees add about 0.03 AUC over a linear model.

Into the HMM. The model's log-odds enter at weight 0.2 per frame. The weight is *gated*: full weight when proximity says nothing, zero when a player is clearly on the ball.

4. Dead-ball detector (`possession/deadball.py`, `possession/tagging.py`)

Log-odds of dead vs live, summed per frame from four sources:

- **Ball seen out of bounds:** +4.
- **Collective speed:** a 2-component Gaussian mixture fitted **without labels** to log crowd speed. It found a slow component at 0.66 m/s and a fast one at 1.54 m/s.
- **Ball rolling inside the pitch / ball unseen:** −0.7 and +0.15. Checked against the tags: the ball is unseen in 35% of tagged-dead frames vs 18% of tagged attacking-phase frames, so "unseen" really is evidence of a stoppage.
- **Set pieces:** dead before the estimated restart, live just after it.
- **Finding the restart.** We measured the tag windows (aggregates only). Kickoffs are ~5 s windows, but every other set piece is a fixed 35 s window in which the players speed up only 20–30% of the way in. So the restart is **estimated per set piece** as the moment of the biggest rise in collective speed. Our first version used the window's midpoint, which marked ~10 s of live play after each set piece as dead. Fixing it raised the validation score from 0.697 to 0.747 without the shape model, and from 0.715 to 0.780 with it.

5. Smart Tagging phases, with self-calibrated direction

Each phase code adds at most ±0.25 log-odds per frame towards its tagged team. The **sign is learned**: on frames where proximity is confident, how often does the tagged team have the ball? Every `DEFENDING …` code and `DEFENSIVE TRANSITION` came out negative (agreement 0.11–0.38), so the tag names the team *without* the ball. `BUILD UP` and `PROGRESSION` came out at 0.86–0.89 agreement. No assumption about what the codes mean was needed.

6. Decoder (`possession/decode.py`)

- **States:** H, A, D, decoded with Viterbi per half.
- **Emissions:** team evidence only decides *which* team. It never makes "alive" look more likely than "dead"; that is left to the dead-ball detector. (Measured before the restart fix, this raised the validation score from 0.608 to 0.697.)
- **Transition costs**, in log-likelihood units:

transition	cost	cheaper where
H ↔ A	14	never. A touch, a deflection or a lost aerial duel cannot pay for it
live → dead	7	1.5 near a ball seen out
dead → live	7	2 within 3 s of an estimated restart

- **Half boundaries:** half 1 starts live, biased to the tagged kickoff side. Half 2 is dead until the published kickoff (frame 113,742, FAQ.md).
- **Clean-up:** possessions under 0.8 s and stoppages under 1.2 s are absorbed into their neighbours.

Possession rules as implemented

- **Loose or contested ball:** no team evidence, so the last team keeps it.
- **Change of possession:** needs *sustained* control by the other side. About 5+ frames of strong proximity, or a longer stretch of shape evidence, pays the switch cost.
- **Restarts:** the set piece anchors the moment play resumes. The side comes from the tag where it names one, otherwise from proximity. Throw-ins are tagged `N/A`.
- **Dead ball:** always `D`. We never assign a team to a dead ball.

How settings were chosen: ball-masking validation (`possession/validate.py`)

We could not tune on the ground truth, so we built a test that needs none.

1. Take frames whose answer is known from the data (the pseudo-labels).
2. Hide the ball in 2–6 s blocks around them: 15% of graded frames, half centred on team frames and half on dead frames, with a fixed seed.
3. Re-run the whole pipeline. The shape model, trained cross-half on unmasked data, never saw the half it predicts.
4. Score **balanced recovery**: the mean of "right team" on hidden team frames and "dead" on hidden dead frames.

All rows below include the per-set-piece restart estimate.

configuration	balanced	team	dead	possessions
no shape model, v1 emissions	0.699	0.566	0.832	274
no shape model, team-only emissions	0.747	0.524	0.971	276
+ team-shape model, weight 0.2 (chosen)	**0.780**	0.588	0.972	281
+ team-shape model, weight 0.4	0.781	0.590	0.972	311
+ dead-shape model, weight 0.05	0.761	0.628	0.894	282
+ dead-shape model, weight 0.2	0.708	0.685	0.730	301

Reading the table:

- With the ball hidden, the shape model cuts **wrong-team** calls from 13.3% to about 5%.
- Weights 0.2 and 0.4 recover equally well, but 0.4 produces 30 more possessions. We took the more conservative 0.2.
- The remaining team-frame errors are calls of `D` during live play with the ball hidden. This is the hardest case: a slow build-up looks like a stoppage.

Tried and dropped:

- **A ball-free dead-ball shape model** (AUC 0.82–0.85 cross-half): it traded dead accuracy for team accuracy and lowered the balanced score at every weight tried.
- **Snapping boundaries to detected touches** (`--snap`): we have no ground-truth-free way to check it, and on the synthetic match it made no difference (0.453 vs 0.464 spellF1, inside the noise band).
- **Writing `X` mid-possession** (`--abstain`): the scorer forward-fills `X`, so spellF1 cannot change, while macroF1, the tie-break, drops (−0.06 on the synthetic match).

Result

- **Output:** 301 possessions, mean 14.8 s, 30% of graded frames `D`.

- **Public 5 minutes:** spellF1 0.42 vs 0.33 for the published floor `baseline_smooth50`, and macroF1 0.68 vs 0.62. This is a correctness check only: 17 possessions cannot rank anything.

Limitations, and what we would do next

- **Live vs dead with the ball hidden** is the weakest part: ~40% of hidden live frames come out `D`. A better dead model would need cleaner pseudo-labels for fouls and stoppages that have no tag.
- **The fixed priors** (switch cost, stop and restart costs) were not tuned. The masking validation measures recovery, not over-switching, so tuning them with it would be unsafe.
- **The broadcast feed** could fill ball gaps if it were aligned to the tactical timeline. We did not attempt that.
- **Pseudo-labels inherit the proximity rule's blind spots** (balls in the air, crowded boxes). We kept them strict for that reason.

Synthetic data

`synthetic/make_synthetic.py` simulates a full match in the bundle's exact file formats, with its own ground truth. We used it to build and debug the pipeline before the gated data arrived, because the competition terms forbid sharing the match data with cloud tools. It is not a model of the real match and was not used to choose any setting.

Data handling

`data/`, `synthetic_data/` and `.cache/` are gitignored. The pipeline prints only counts, shares, fitted aggregates and model metrics, never tracking rows.